AIA9: Reinforcement Learning

Module Introduction and Theoretical Foundations

Reinforcement Learning (RL) trains agents through **trial-and-error interaction** with environments, powering AlphaGo, ChatGPT alignment (RLHF), and autonomous robotics. This module covers foundational theory (MDPs, Bellman equation), modern deep RL algorithms (PPO, SAC), and RLHF for LLM alignment.

Core Mathematical Framework:

text

MDP = (S, A, P, R, γ)

Policy π(a|s): Behavior function

Value V^π(s): Expected discounted return

Q^π(s,a): Action-value function

RL Taxonomy:

text

Model-Free (DQN, PPO) vs Model-Based (MuZero)

On-Policy (PPO) vs Off-Policy (SAC, DDPG)

Actor-Critic vs Value-Based

(Figure 1: RL algorithm landscape)

Section 1: Markov Decision Processes and Bellman Equation

Formal MDP Definition:

text

S: State space A: Action space

P(s'|s,a): Transition R(s,a,s'): Reward

γ ∈ [0,1): Discount factor

Bellman Equations:

Value Function:

text

V^π(s) = E[Σ γ^t r_t | s_0=s, π]

= E[r(s,π(s)) + γ V^π(s') | s]

Q-Function:

text

Q^π(s,a) = E[r(s,a) + γ V^π(s')]

V^π(s) = max_a Q^π(s,a)

Policy Improvement:

text

π_new(s) = argmax_a Q^π(s,a)

Policy Iteration Algorithm:

text

1. Policy Evaluation: V^π ← T^π V (Bellman backup)

2. Policy Improvement: π ← greedy(V)

Converges in finite steps (MDP)

Section 2: Value-Based Methods - Q-Learning and DQN

Q-Learning (Watkins 1989):

text

Q(s,a) ← Q(s,a) + α [r + γ max_a' Q(s',a') - Q(s,a)]

Off-policy TD(0)

Deep Q-Networks (Mnih 2013):

text

1. Experience replay buffer

2. Target network Q_target = θ_target

3. Double DQN: argmax_a Q(s',a;θ_target) sample Q(s',a;θ)

Complete DQN Implementation:

python

```python
import torch
import torch.nn as nn
import numpy as np
from collections import deque
import random


class DQN(nn.Module):
    def __init__(self, state_dim, action_dim, hidden_dim=256):
```

```
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(state_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, action_dim)
        )

    def forward(self, x):
        return self.net(x)

class DQNAgent:
    def __init__(self, state_dim, action_dim, lr=1e-3, gamma=0.99,
                 buffer_size=10000, batch_size=32):
        self.q_net = DQN(state_dim, action_dim)
        self.target_net = DQN(state_dim, action_dim)
        self.optimizer = torch.optim.Adam(self.q_net.parameters(), lr=lr)

        self.gamma = gamma
        self.replay_buffer = deque(maxlen=buffer_size)
        self.batch_size = batch_size
        self.epsilon = 1.0
        self.epsilon_decay = 0.995
        self.epsilon_min = 0.01

    def select_action(self, state):
        if np.random.rand() < self.epsilon:
            return random.randrange(self.q_net.net[-1].out_features)

        q_values = self.q_net(torch.FloatTensor(state))
```

```
    return q_values.argmax().item()

def store_transition(self, state, action, reward, next_state, done):
    self.replay_buffer.append((state, action, reward, next_state, done))

def train_step(self):
    if len(self.replay_buffer) < self.batch_size:
        return

    batch = random.sample(self.replay_buffer, self.batch_size)
    states, actions, rewards, next_states, dones = zip(*batch)

    states = torch.FloatTensor(np.array(states))
    next_states = torch.FloatTensor(np.array(next_states))
    actions = torch.LongTensor(actions)
    rewards = torch.FloatTensor(rewards)
    dones = torch.BoolTensor(dones)

    # Current Q values
    q_values = self.q_net(states).gather(1, actions.unsqueeze(1))

    # Target Q values (Double DQN)
    next_actions = self.q_net(next_states).argmax(1, keepdim=True)
    target_q = self.target_net(next_states).gather(1, next_actions)
    targets = rewards + self.gamma * target_q.squeeze() * (~dones)

    # Loss
    loss = F.mse_loss(q_values.squeeze(), targets)

    self.optimizer.zero_grad()
    loss.backward()
```

```
        self.optimizer.step()

        # Epsilon decay
        self.epsilon = max(self.epsilon_min, self.epsilon * self.epsilon_decay)

        return loss.item()

    def update_target(self):
        self.target_net.load_state_dict(self.q_net.state_dict())
```

Section 3: Policy Gradient Methods and REINFORCE

Policy Gradient Theorem:

text

$\nabla J(\theta) = E[\ \Sigma_t\ \nabla \log\pi(a_t|s_t;\theta)\ A_t\ ]$

Advantage $A_t = Q(s_t,a_t) - V(s_t)$

REINFORCE Algorithm:

text

1. Rollout trajectory $\tau \sim \pi_\theta$
2. Return $R(\tau) = \Sigma\ \gamma^t\ r_t$
3. $\nabla J \approx \Sigma_t\ \nabla \log\pi(a_t|s_t;\theta)\ (R_t - b)$

b: baseline (reduces variance)

Actor-Critic Methods:

text

Actor: $\pi_\theta(a|s)$

Critic: $V_\phi(s) \approx$ value function

Section 4: Proximal Policy Optimization (PPO)

PPO (Schulman 2017) - Industry Standard:

text

Trusted region policy optimization

Clip objective prevents destructive updates

Clipped Surrogate Objective:

text

L_clip = min(r(θ) Â, clip(r(θ), 1-ε, 1+ε) Â)

r(θ) = π_θ(a|s) / π_old(a|s)

Complete PPO Implementation:

python

```
class PPOAgent:
    def __init__(self, state_dim, action_dim, hidden_dim=256,
                 clip_eps=0.2, epochs=4, gamma=0.99):
        self.actor = ActorNet(state_dim, action_dim, hidden_dim)
        self.critic = CriticNet(state_dim, 1, hidden_dim)
        self.actor_optimizer = torch.optim.Adam(self.actor.parameters(), lr=3e-4)
        self.critic_optimizer = torch.optim.Adam(self.critic.parameters(), lr=3e-4)

        self.clip_eps = clip_eps
        self.epochs = epochs
        self.gamma = gamma

    def compute_gae(self, trajectory):
        """Generalized Advantage Estimation"""
        rewards, values, dones = trajectory.rewards, trajectory.values, trajectory.dones

        advantages = []
        gae = 0
        for t in reversed(range(len(rewards))):
            if t == len(rewards) - 1:
                next_value = 0
            else:
                next_value = values[t+1]

            delta = rewards[t] + self.gamma * next_value * (1-dones[t]) - values[t]
```

```
        gae = delta + self.gamma * 0.95 * (1-dones[t]) * gae
        advantages.insert(0, gae)

    return advantages

def ppo_update(self, trajectories):
    for epoch in range(self.epochs):
        for trajectory in trajectories:
            states = torch.FloatTensor(trajectory.states)
            actions = torch.LongTensor(trajectory.actions)
            old_logprobs = torch.FloatTensor(trajectory.logprobs)
            advantages = torch.FloatTensor(self.compute_gae(trajectory))
            returns = advantages + trajectory.values

            # Actor loss
            logits = self.actor(states)
            dist = Categorical(logits=logits)
            new_logprobs = dist.log_prob(actions)
            ratio = (new_logprobs - old_logprobs).exp()

            surr1 = ratio * advantages
            surr2 = torch.clamp(ratio, 1-self.clip_eps, 1+self.clip_eps) * advantages
            actor_loss = -torch.min(surr1, surr2).mean()

            # Critic loss
            values = self.critic(states).squeeze()
            critic_loss = F.mse_loss(values, returns)

            # Update
            self.actor_optimizer.zero_grad()
            actor_loss.backward()
```

```
        nn.utils.clip_grad_norm_(self.actor.parameters(), 0.5)
        self.actor_optimizer.step()

        self.critic_optimizer.zero_grad()
        critic_loss.backward()
        nn.utils.clip_grad_norm_(self.critic.parameters(), 0.5)
        self.critic_optimizer.step()
```

Section 5: RLHF - LLM Alignment

RLHF Pipeline (Ouyang 2022):

text

1. SFT: Supervised fine-tuning
2. Reward Model: Rank preferences → Bradley-Terry
3. PPO: Optimize policy vs reward model + KL penalty

Reward Model Training:

text

$P(y1 > y2 \mid x) = \sigma(r_\phi(x,y1) - r_\phi(x,y2))$

$\text{Loss}: -\log \sigma(r(y_w) - r(y_l))$

PPO for Language:

text

$\text{Reward} = r_model(x,y) - \beta\, KL(\pi \mid\mid \pi_ref)$

DPO Alternative (Rafailov 2023):

text

Direct optimization: No RL needed!

$L_DPO = \log \sigma(\beta \log(\pi(y_w)/\pi_ref) - \log(\pi(y_l)/\pi_ref))$

Section 6: Model-Based RL and Advanced Topics

MuZero (Schrittwieser 2020):

text

Learn model, value, policy simultaneously

MCTS planning with learned dynamics

World Models (Ha 2018):

text

VAE + RSSM + Controller

Dream to learn

Offline RL:

text

Conservative Q-Learning (CQL)

Decision Transformer

Section 7: Production RL Systems

RL Production Checklist:

text

1. Reward engineering (shaping, sparse → dense)

2. Sample efficiency (experience replay, HER)

3. Stability (target networks, clipping)

4. Safety (constrained policy optimization)

Case Studies:

1. **AlphaStar**: 1M simultaneous agents
2. **ChatGPT RLHF**: Human preference alignment
3. **AlphaCode**: Competitive programming

Capstone: Custom RL Environment + PPO

Trading agent, game AI, or robotic control